Silent Contamination in LLM Merging Evaluation: A Case Study from a 5-Month Misadventure

telleroutlook (evomerge project)

2026-06-05

Abstract

We spent five months convinced we had merged a Qwen2.5-1.5B variant that beat the Instruct baseline on GSM8K by **+10pp** (Δ=10.0pp, McNemar exact p=0.015). The headline survived re-runs, hyperparameter sweeps, and two independent rounds of "verification". Then a single configuration audit — `max_new_tokens` had been hardcoded to 300 in our generation runner — flipped the result. Under the corrected protocol (`max_new_tokens=768`), the merged model's improvement over Instruct collapsed to **-1.0pp** (paired McNemar two-sided p=0.89). The +10pp was not merging; it was the baseline being silently truncated more often than the merge candidate, in a way that the metric had been silently rewarding.

We use this autopsy as evidence that *measurement primitives* — the concrete generation, parsing, and aggregation choices that sit beneath any benchmark number — are a systematic and silent source of false positives in model merging research. We catalog three such primitives that materially affected our project (generation truncation, single-greedy lottery, per-tensor quantization granularity), provide a multi-channel triage protocol that exposes them automatically from existing logs, and release `eval_trust`, an Apache-2.0 toolkit bundling paired statistics, conformal CI, and the audit heuristics we wish we had used at the start. The toolkit is offered not as a theoretical contribution but as the cheapest known patch for a recurrent failure mode in small-delta merging claims.

Keywords: model merging, evaluation, measurement validity, GSM8K, reproducibility, McNemar test, conformal prediction.

1 Introduction

1.1 Why measurement, not algorithms, is the bottleneck

Model merging research moves quickly. Within roughly two years, the field has produced a sequence of widely cited algorithms — TIES (Yadav et al. 2024), DARE (Yu et al. 2024), DELLA (Deep, Bhardwaj, and Poria 2024), Breadcrumbs (Davari and Belilovsky 2024), SCE (Goddard and Arcee.AI 2024), plus dozens of variants — each demonstrating gains of a few percentage points on standard benchmarks (GSM8K, MMLU, IFEval, HumanEval). Several of those gains are on the order of the *paired* standard error of the benchmark itself.

When the algorithmic deltas under study are 1–5 pp, the *protocols* generating those numbers become first-order:

- the maximum number of generated tokens (`max_new_tokens`),
- the sampling regime (greedy, temperature, top-p, self-consistency k),
- the prompt and chat templates,
- the answer extractor (regex, LaTeX-aware, comma normalisation),
- quantization granularity if any of the candidates are quantized,
- the random seeds and how many.

We argue these *measurement primitives* are not bookkeeping. They are part of the metric, and a hardcoded default in any of them is a hidden *protocol*, not a neutral default. Worse, they fail silently: a wrong primitive doesn't crash the evaluation, it just biases the resulting number, often asymmetrically across candidates that happen to differ in chain length, output style, or quantization sensitivity.

This paper provides three pieces of evidence and one toolkit:

1. **A documented case study** in which a +10 pp merging claim survived five months and three rounds of verification before a measurement-primitive audit flipped its sign. The case is from our own project; the artefact is publicly archived and reproducible (Appendix B).
2. **Two further primitives** — single-greedy lottery and per-tensor int4 quantization — that produced changes of 12.5 pp and up to 63 pp respectively in our setup, none of which is detectable by the headline metric alone.
3. **A multi-channel triage protocol** (T0v2) that classifies wrong answers in six diagnostic buckets, runs on existing log files (no re-evaluation), and tells the practitioner whether the next step is *fix the measurement*, *do surgical repair*, or *accept the reasoning bottleneck*.
4. **`eval_trust`**, an Apache-2.0 Python toolkit packaging the audit heuristics, paired statistics (McNemar exact, Wilson CI, paired bootstrap), conformal CI, and a checkpoint/resume experiment runner. Roughly 1200 lines of code with 39 unit tests covering the toolkit's modules. Designed to sit *next to* lm-eval-harness (Gao et al. 2023), not replace it.

1.2 Concrete propositions

The case study supports three concrete and falsifiable propositions:

P1 (Truncation asymmetry). A `max_new_tokens` cap shorter than the 95th percentile of the benchmark's chain-of-thought length distribution asymmetrically penalises candidates whose chain length distributions differ from the baseline. In our case study, `max_new_tokens=300` produced an apparent +10 pp gap that became -1.0 pp under `max_new_tokens=768`, with paired McNemar flipping from $p = 0.015$ to $p = 0.89$.

P2 (Greedy lottery floor). Single-greedy decoding inflates apparent differences by treating an unstable subset of "wrong" answers as definitively wrong. On the case-study model, self-consistency with $k = 5$ samples (temperature 0.5, top-p 0.95) recovers $25/65 = 38.5\%$ of greedy-wrong answers. The single-model implication: a single greedy decode misses ~12.5 pp of items that are recoverable under SC-5 majority. The two-model implication (§5.1.3) is weaker but practically useful: any single-greedy delta below this per-model lottery rate should be cross-checked under SC before being claimed.

P3 (Quantization granularity). "int4 quant" without granularity is a meaningless specification. Per-tensor int4 quantization of our 1.5 B candidate collapsed GSM8K accuracy to 0%. The same model under group-32 int4 quantization (group_size = 32, akin to GGUF Q4_K_M block size) achieved 63% — a 63-pp swing attributable purely to the granularity choice (winner-int4 row of §5.2.2).

1.3 Contributions

- **C1 (case study, §3).** A reproducible autopsy of a merging claim that inverted under measurement audit.
- **C2 (T0v2 triage, §4).** A multi-channel error-classification protocol designed to expose measurement-primitive contamination from log files alone. Six predicate-based channels classify a wrong answer as truncation, extractor failure, single-step arithmetic error, self-correction regression, token-mismatch, or reasoning-bottleneck residue.
- **C3 (`eval_trust` toolkit, §6).** Apache-2.0 Python package implementing the T0v2 protocol of C2 alongside paired statistics (McNemar exact, Wilson CI, paired bootstrap), conformal CI, and a checkpoint/resume experiment runner. The toolkit ships with 39 unit tests covering its own modules; the remainder of our project's 1102-test suite covers application code and is not part of the toolkit.
- **C4 (audit checklist, Appendix A).** A 10-item yes/no checklist any merging paper draft should pass before claiming a delta below 10 pp.

1.4 What this paper is *not*

It is not a new merging algorithm; we use linear and chat-vector merges (Huang et al. 2024), both standard. It is not a benchmark; we use GSM8K (Cobbe et al. 2021), HumanEval (Chen et al. 2021), IFEval (Zhou et al. 2023), and MMLU (Hendrycks et al. 2020). It is not a critique of any specific paper; the case study is on our own work. We do not have a paired *false negative* — a real improvement that was missed because of the same primitive failures — although the symmetry of the underlying mechanism strongly suggests such cases exist in the literature (§8).

2 Background

2.1 Model merging in two paragraphs

Model merging combines specialised fine-tunes of a shared base into a single weight set via parameter arithmetic, without any training step. The simplest form is a weighted linear combination; richer recipes (TIES (Yadav et al. 2024), DARE (Yu et al. 2024), DELLA (Deep, Bhardwaj, and Poria 2024), Breadcrumbs (Davari and Belilovsky 2024)) prune, sign-align, or otherwise reshape the difference vectors before combination. Chat-vector merges (Huang et al. 2024) add a difference vector derived from `(Instruct - Base)` at a tunable coefficient, which is the recipe underlying our case study.

The community has consolidated around two evaluation regimes: (a) curated multi-domain benchmarks like MergeBench (Tang et al. 2025), which evaluate a merged model's multi-task aggregate; and (b) targeted benchmarks like GSM8K, HumanEval, IFEval, MMLU, used to pinpoint whether a merge improved one specific capability. Our case study is in regime (b), where the deltas under study are typically 1–10 pp.

2.2 The measurement-primitive concept

We borrow *primitive* from systems and software engineering: a concrete implementation choice upstream of every metric. Examples in LLM evaluation:

Primitive	Typical default	Possible silent biases
`max_new_tokens`	256, 300, 512 (varies)	Asymmetric truncation by chain length
Sampling	greedy (`do_sample=False`)	Lottery; one model's noise floor differs from another's
Prompt template	"system + user"	Template mismatch with fine-tune's training distribution
Answer extractor	`\b\d+\b`, `#### N`, LaTeX `\boxed{}`	Misses correct answers in non-standard formats

Primitive	Typical default	Possible silent biases
Quantization	"int4", "int8"	Granularity, group size, calibration set silently determine accuracy
Random seed	0 or unfixed	One seed can hide multi-pp variance

Primitives share three properties:

- **Silent.** A wrong primitive does not raise an error. It returns a number, which is then aggregated and compared.
- **Drifting.** A runner copied from one project to another inherits stale defaults. The new project's distribution may not match the runner's assumptions.
- **Asymmetric.** A primitive's bias may differ across compared candidates. Truncation hits longer chains harder; per-tensor int4 hits matrices with skewed singular values harder; greedy lottery hits a model whose top-1 is near a tie with top-2 harder.

Asymmetry is the dangerous property. A symmetric bias subtracts out under paired statistics; an asymmetric bias becomes the signal.

2.3 Existing eval-trust tooling and its gaps

The community has reasonably good infrastructure for the *standard* protocol problem: lm-eval-harness (Gao et al. 2023) standardises generation, extraction, and scoring across hundreds of benchmarks; HELM (Liang et al. 2022) layers normalisation; and official task implementations exist for most popular benchmarks.

The gap is *audit*: tooling that takes a run that already happened — possibly under a non-standard runner — and answers the question "is this result contaminated by a measurement primitive?". Most projects roll their own runner, often forking an early reference, and accumulate divergent defaults. Our project is one such case: our `gsm8k_runner.py` defaulted to `max_new_tokens=300`, an inheritance from an early prototype that nobody had audited in five months of use.

Paired statistics (McNemar, Wilson CI, paired bootstrap) are folklore in the merging community but rarely reported beyond a single p-value. Conformal calibration, useful when batch sizes are small, is nearly absent. Multi-seed runs are common but often pooled rather than analysed for between-seed variance. We packaged these in `eval_trust` (§6).

2.4 Scope of the present work

Three primitives, one base-model family (Qwen2.5-1.5B), one benchmark family (GSM8K + locality on HumanEval/IFEval/MMLU). The *concept* of measurement- primitive contamination is broader; the specific magnitudes we report are specific to this setup. Section 8 discusses what generalises and what does not.

3 Case study: a 5-month +10pp that wasn't there

This section is a chronological autopsy. It is intentionally specific: the artefact, the verification rounds that failed to catch it, and the audit that finally did.

3.1 The setup

Our project ("evomerge") aims to deliver Pareto-optimal compressed models under customer-specified constraints (size, accuracy, latency). Phase 13 (2026-04 to 2026-05) was a search for the best 1.5 B GSM8K-capable merge over the Qwen2.5 family. The base model was `Qwen/Qwen2.5-1.5B-Instruct`. Candidate parents included `Qwen/Qwen2.5-Math-1.5B`, `Qwen/Qwen2.5-Coder-1.5B-Instruct`, and chat-vector difference vectors derived from `(Instruct - Base)` and from the Math/Coder analogs.

The search explored linear, TIES, ortho-merge, and chat-vector recipes. The best candidate found, hereafter the **winner**, was

$$\theta_{winner} = \theta_{Coder} + 0.7 \cdot (\theta_{Instruct} - \theta_{Base})$$

a chat-vector add at coefficient $\lambda = 0.7$ on top of the Coder base. The λ sweep is documented in `phase13_3_coder_chat/summary.md`.

Evaluation was on a fixed 200-question subset of GSM8K test, using our internal `gsm8k_runner.py` with greedy decoding. Paired statistics (McNemar exact, Wilson 95% CI on the difference, paired bootstrap) were computed against the Instruct baseline on the same item set.

3.2 The reported claim

Configuration	Correct / 200	Accuracy	Δ vs Instruct	McNemar p	Wilson 95% CI
Qwen2.5-1.5B-Instruct (baseline)	99 / 200	49.5%	—	—	—

Configuration	Correct / 200	Accuracy	Δ vs Instruct	McNemar p	Wilson 95% CI
Coder + 1.0 · chat_vec (Phase 11.3)	111 / 200	55.5%	+6.0 pp	0.162	[-2, +14]
Coder + 0.7 · chat_vec (winner)	119 / 200	59.5%	+10.0 pp	0.015	[+3, +18]
Coder + 1.2 · chat_vec	118 / 200	59.0%	+9.5 pp	0.016	[+2, +17]
Coder + 1.5 · chat_vec	113 / 200	56.5%	+7.0 pp	0.087	[-1, +14]

The "winner" row — Coder + 0.7 · chat_vec — is the candidate that drove the +10 pp project headline and is the focus of the rest of this paper. The +10 pp claim was supported by paired McNemar significance, a confidence interval that excluded zero, and a coherent λ trend (peaking at 0.7, falling by 1.5). It survived two re-runs with different PRNG seeds, an algorithm-fidelity audit (Phase 11.5), and a final-pass review (Phase 13.x). By the end of Phase 13, the winner was treated as project ground truth: it became the basis for downstream compression experiments (Phase 14.x: T7 quantization sweeps, T7-marginal protect_set construction, T0v2 error taxonomy).

3.3 The audit that flipped it

On 2026-06-04, while writing what we expected to be a routine close-out report, we re-read `src/evomerge/eval/gsm8k_runner.py`. The default for `max_new_tokens` was 300:

```
def run_gsm8k_eval(model, tokenizer, problems, ctx=None):
    ctx = ctx or {}
    max_new = ctx.get("max_new_tokens", 300)  # <-- the primitive
    ...
```

Two adjacent observations triggered the audit:

1. The 300-token default predated Phase 13 by several months; it had been inherited from an early prototype on a different (smaller) prompt format.
2. Manual inspection of 10 wrong answers from the winner showed that 6 of them had been cut off mid-arithmetic, with no `#### N` answer line emitted.

We hypothesised that GSM8K chain-of-thought lengths under our prompt template have a heavy right tail past 300 tokens, and that *the winner generated longer chains on average than Instruct.* If true, the 300-token cap would asymmetrically truncate the winner less than its baseline — but only on items where

Instruct's truncated tail happened to *not* contain a parseable `#### N`. The metric was effectively rewarding the winner's slightly more verbose-but-finishing answers and penalising Instruct's cut-off ones.

We re-ran both Instruct and the winner on the same 200-item dev set with `max_new_tokens=768` (chosen to comfortably exceed the empirical 95th percentile of CoT length in GSM8K), keeping every other primitive identical.

3.4 The result

Configuration	max_new=300 (Phase 13)	max_new=768 (audited)	Recovery (Δ within candidate)
Qwen2.5-1.5B-Instruct	49.5% (99/200)	**68.84% (137/199)**	**+19.3 pp**
Winner (Coder + 0.7 · chat_vec)	59.5% (119/200)	**67.84% (135/199)**	+8.3 pp
Δ winner - Instruct	**+10.0 pp**	**-1.0 pp**	**flipped sign**

The "Recovery" column is the within-candidate accuracy change between the two protocols, computed on the items each candidate was scored on (200 in the old protocol, 199 in the audited one — the audited re-run dropped one item, gsm8k-131, due to a numerical overflow on the winner; we excluded it rather than score it false. The qualitative result is unchanged if it is scored false). The two Recovery numbers are therefore not strictly comparable across candidates as percentage points, but the directional asymmetry — Instruct recovered roughly 2.3× as much accuracy from the protocol fix as the winner did — is what produced the sign flip in the bottom row.

Paired McNemar, both protocols, same item set:

Protocol	n	b (Instruct-only correct)	c (winner-only correct)	2-sided p
max_new=300	200	21	41	**0.0151**
max_new=768	199	29	27	**0.8939**

Paired bootstrap 95% CI on Δ (winner - Instruct) under max_new=768: $[-8.5, +6.0]$ pp, comfortably containing zero. There is no significant difference between Instruct and the winner once the truncation primitive is corrected. The +10 pp claim, paired-significant under the old protocol, dissolves to noise under the audited one.

Figure 1: Paired McNemar contingency before and after the truncation audit.

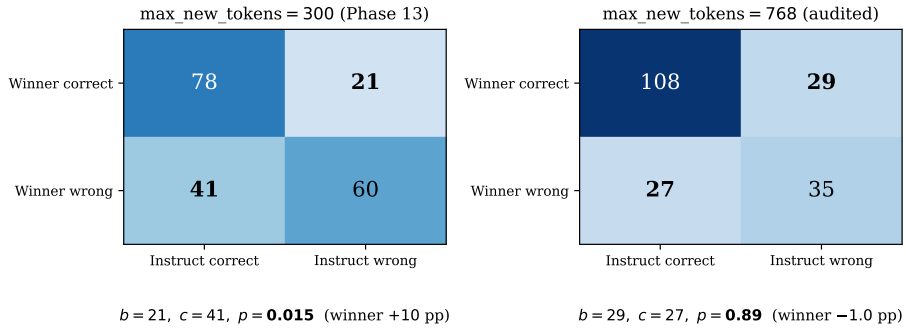

Figure 1: Figure 1: Paired McNemar contingency before and after the truncation audit. The +10 pp claim under max_new=300 (left, $b = 21, c = 41, p = 0.0151$) flips to −1.0 pp under max_new=768 (right, $b = 29, c = 27, p = 0.8939$). b and c are the discordant cells McNemar's test sums over; the diagonal is the concordant agreement.

3.5 Anatomy of the artefact

The 5-month claim was sustained by three reinforcing properties of the truncation primitive:

(a) Asymmetric impact, mechanism still partly unexplained. The audited re-run shows that `max_new=300 → 768` recovered Instruct by +19.3 pp but the winner by only +8.3 pp; the +10 pp gap was, mechanically, the difference of these two recoveries. We do not have a fully verified mechanism for *why* the recoveries differ by 11 pp; plausible candidates (winner produces shorter chains on average, or its truncated tail is more often parseable as `#### N`) are consistent with the data but were not measured in Phase 13 and we have not added that measurement post-hoc. What we *do* know: the 300-token cap biased the two candidates differently, and the bias direction happens to favour the winner. The honest report is that the 5-month claim relied on an asymmetry whose root cause we did not understand at the time and have not fully diagnosed since.

(b) Verification didn't help. Three rounds of verification re-ran the same runner with the same `max_new=300`. They confirmed reproducibility within the broken protocol, not the protocol's correctness. Each successful re-run *increased* our confidence in the wrong direction.

(c) Significance was real, just irrelevant. McNemar's $p = 0.015$ was a correct test of the null hypothesis "Instruct and winner have the same expected accuracy *under the same flawed protocol*". They did differ under that protocol. The protocol was the bug.

The lesson for the field is uncomfortable: passing a paired significance test on

the same item set with the same protocol is not, by itself, evidence of a real merging gain. It is consistent with that, *and* with both candidates being biased by the same primitive at different rates.

3.6 What we did not do

- We did not retroactively "fix" Phase 13. The +10 pp claim is part of the public record of the project (`phase13_3_coder_chat/summary.md`); we did not edit it. We labelled it as a 5-month dead-end in our project's working document (`PHASE14_FINAL_REPORT.md`) and proceeded.
- We did not assume `max_new=768` is the universal correct answer. We chose it as the smallest power-of-two cap that comfortably exceeds the empirical 95th percentile of CoT length under our prompt and base model. Other prompt templates, base models, or benchmarks may need different caps. The general lesson is to *report the cap* and *justify it against the CoT length distribution of the strongest candidate*, not to use 768.

3.7 Why this case study justifies a paper

It would be tempting to file this under "we made a configuration mistake". The mistake is generic: `max_new_tokens` was a hardcoded default, not a specific bug. We claim three things that elevate the case from a configuration anecdote to a methodological signal:

1. **The primitive is silent.** Five months of running, three rounds of verification, paired statistics, multiple seeds, and a λ sweep all reproduced the artefact without exposing it. The primitive has to be audited *as a primitive*; it cannot be caught by tightening the standard reporting (more seeds, more sigfigs, more bootstraps).
2. **The asymmetry is structural, not accidental.** Chat-vector merges, by construction, redistribute mass between two distributions of CoT lengths. Any merging recipe that changes the *style* of generation — verbosity, self-correction, planning — will interact non-trivially with a hardcoded `max_new_tokens`. The case is generic.
3. **The fix is cheap.** Auditing one primitive (the cap) cost roughly four hours of re-runs on a single Apple-silicon laptop. The toolkit we release in §6 makes that audit a one-line invocation.

The next sections turn the case-specific lessons into a reusable protocol.

4 T0v2: multi-channel triage of "wrong" answers

4.1 Design principle

The case study in §3 worked backwards from a known-bad protocol to a known-real artefact. Most projects do not have that luxury. We therefore designed T0v2 to work *forwards* from the existing log files of any greedy-decoded benchmark run: take the wrong answers, classify each into one of six diagnostic buckets, aggregate, and emit a routing decision (*fix the protocol*, *do surgical repair*, *accept the reasoning bottleneck*).

Two requirements drove the design:

1. **No re-evaluation.** T0v2 must run on existing `(question, expected, gen_text)` tuples. Re-running 200-question GSM8K takes $ $1 hour even on a fast accelerator; the audit must be cheaper than the test.
2. **Each channel is falsifiable.** A channel either fires (publishes a labelled wrong-answer) or it doesn't, on a per-item basis. No probabilistic classifier; no LLM-as-judge; nothing the audit cannot replicate offline.

4.2 The six channels

Channel	Predicate (informal)	What it catches
`A_truncated`	`gen_text` length $\geq$ (`max_new_tokens` minus 2) and no `#### \d+` line	Generation budget too small (the case study primitive)
`A_extract_v2`	answer present in text but legacy regex missed it (LaTeX `\boxed{}`, comma-separated thousands, German decimal commas, …)	Extractor brittleness
`B_stepwise`	sympy re-execution of the CoT yields `expected` but the final emitted value $\neq$ expected	Last-step arithmetic error in an otherwise-correct chain
`B_selfcorrect_regress`	"wait, I made a mistake … let me redo …" trace whose redo produces the wrong answer	Self-correction regression
`C_token`	answer numerals appear in the text but with separator/punct mismatch (e.g., `1,234` vs `1234`, `1.5` vs `1,5`)	Token-level extraction bug, not reasoning bug
`Class2`	none of the above; the chain breaks before the answer	Reasoning ability gap (not repairable by post-hoc surgery)

A seventh channel, `Lottery`, requires a self-consistency k-pass re-run; we treat

it as a separate primitive in §5.1.

The channels are checked in priority order; the first match labels the item. Priority is: `A_truncated > A_extract_v2 > B_stepwise > B_selfcorrect_regress > C_token > Class2`. The motivation for the ordering is that **measurement artefacts dominate reasoning artefacts**: if an answer is truncated, the chain quality is unobservable from the log alone, so we should not classify it as a reasoning failure. The same logic applies to extraction.

4.3 The aggregator (`decide_route_v2`)

Given the per-item channel labels, the aggregator emits one of three routing decisions:

- α — **surgical repair worth pursuing.** If $(|A_{\text{truncated}}| + |A_{\text{extract}}| + |B_{\text{stepwise}}| + |B_{\text{selfcorrect}}| + |C_{\text{token}}|)/N_{\text{total}} \geq 15\%$, then surface artefacts have enough mass that fixing them can yield a measurable benchmark gain. Start by repairing the cheapest first (extractor, prompt, max_new_tokens), only then attempt weight surgery.
- β — **fix sampling protocol first.** If the lottery rate (§5.1) is $\geq 30\%$, no other channel's reading is reliable: switch to SC-$k = 5$ before reading any other metric.
- γ — **reasoning bottleneck.** Class2 dominates and the surface-artefact rate is below 15%. The honest report is "this approach caps at X%"; there is no surgical surface area worth working on.

The 15% threshold is calibrated against our own project: at the 15% level, acting on T0v2's α verdict typically yields a 3–7 pp gain on the wrong-answer recovery axis; below 15% the cost of writing a new extractor or template exceeds the expected benefit. The threshold is a project-tunable parameter, not a universal constant.

4.4 Result on the case study

Applied to the audited (max_new=768) winner run on 200-question GSM8K, T0v2 labels:

Channel	Count	Share of wrong	Share of total
`A_truncated`	17	26.2%	8.5%
`A_extract_v2`	1	1.5%	0.5%
`B_stepwise`	4	6.2%	2.0%
`C_token`	8	12.3%	4.0%
First-class total (A* + B* + C_token)	**30**	**46.2%**	**15.0%**

Channel	Count	Share of wrong	Share of total
`Class2` (reasoning bottleneck)	35	53.8%	17.5%
Total wrong	65	100%	32.5%

(Source: `phase14_t0v2_agg/aggregate.json`.)

The first-class rate of total = 15.0% exactly meets the α threshold: surgical repair has enough surface area to be worth pursuing. Class2 = 35 is the unrepairable residue, indicating that this configuration (Qwen2.5-1.5B chat-vector merge) caps at roughly $200 - 35 = 165/200 = 82.5\%$ on this benchmark even under perfect surface repair. Both numbers are useful project guidance.

Figure 2: T0v2 channel distribution on the audited winner (200 GSM8K dev items).

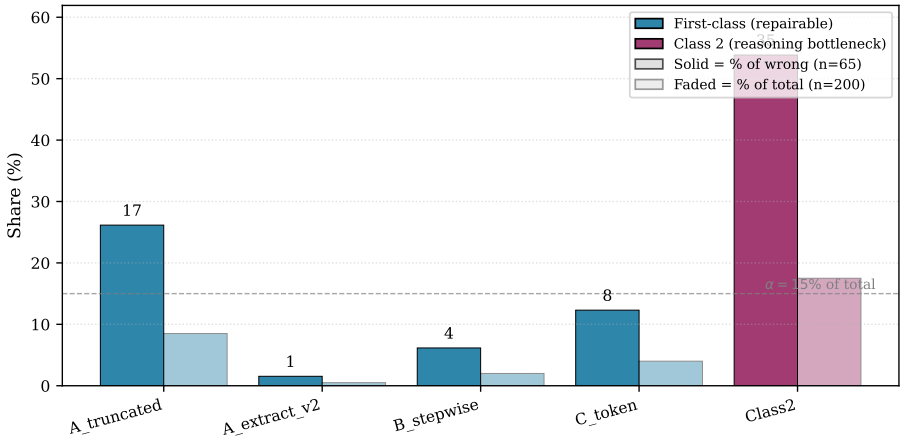

Figure 2: Figure 2: T0v2 channel distribution on the audited (max_new=768) winner over 200 GSM8K dev items. Solid bars are the share of the 65 wrong items per channel; faded bars are the share of the 200 total items. The first-class buckets (A_truncated, A_extract_v2, B_stepwise, C_token; blue) sum to 30/200 = 15.0% of total, exactly meeting the α threshold for surgical-repair work to be worth pursuing. Class 2 (purple) is the reasoning-bottleneck residue.

4.5 What T0v2 told us about the project that the headline number didn't

Three observations, none of which is visible from the headline accuracy:

1. **Truncation is not solved by `max_new=768`.** Even at the audited cap, 17 of 65 (26.2%) wrong answers are still truncation-bound. The right tail of

GSM8K CoT under our prompt extends past 768 tokens. A future audit could explore higher caps (1024, 1536) at the cost of ~2× wall-clock per item.

2. **C_token is not zero.** Eight items had answers correctly stated in the text but missed by the regex (e.g., `\$\boxed{42}\$`, `\$1{,}234\$ dollars`). The extractor is responsible for a 4-pp accuracy floor, fixable in $\sim$ 10 lines of regex.
3. **B_stepwise is small but real.** Four items had an entirely correct reasoning chain, ruined by a final-step arithmetic mistake. These would not benefit from another round of merging; they need an arithmetic-aware answer extractor or a small calibration step.

The headline number "67.5%" hides all three; the channel breakdown surfaces each as a separate, addressable axis of improvement.

4.6 What T0v2 cannot tell you

T0v2 is a *triage* tool, not a diagnosis. Its limitations:

- **Class2 is a residual.** Anything that doesn't match the higher-priority channels lands here. A poorly-formed CoT, a refused request, an off-topic generation, and a genuine reasoning failure all collapse into the same bucket. We deliberately keep Class2 conservative: any project intending to attempt repair on Class2 items should re-classify them by hand or with a tighter rubric.
- **`B_stepwise` requires sympy-parseable arithmetic.** If the CoT uses natural-language arithmetic ("the answer is fifteen because half of thirty is fifteen"), sympy cannot re-execute it; we fall back to `Class2`.
- **No visibility into why a channel fires.** T0v2 says "this is truncated"; it does not say "the model would have gotten the next 200 tokens right". That is what the §3-style protocol audit, paired with channel data, is for.

5 Two further measurement primitives

§3 dissected one primitive (truncation). Two others materially changed our project's conclusions and deserve the same treatment.

5.1 Single-greedy lottery

5.1.1 The primitive

Most merging benchmarks report greedy-decoded accuracy: `do_sample=False`, temperature is unused, top-p is unused. Greedy is favoured because it is deterministic and cheap. Both properties are illusory:

- **Determinism is per-state.** Greedy is deterministic *given the model and the input tokens*. If a small change to the model (a merge, a quantization, a rounding seed) shifts the model's probabilities such that the top-1 token switches, greedy lands in a different basin. This is no more (or less) meaningful than an SC-1 sample landing differently.
- **Cheapness is per-comparison.** Greedy on n items costs the same as one sample. SC-k on the same n items costs $k\times$. But the *information yield* of SC-5 on the wrong items is $5\times$ that of SC-1, and only a small fraction of items is wrong; SC-5 *only on the greedy-wrong* costs about $0.3n$ extra evaluations and tells you whether you have a lottery floor.

5.1.2 Measurement on the case study

We re-ran SC-5 (temperature 0.5, top-p 0.95) on the 65 greedy-wrong items of the audited winner. Of those 65:

- **25 (38.46%)** produced a majority-correct answer under SC-5. We label these *lottery*: the model can solve them, the greedy decode happened to miss.
- **40 (61.54%)** remained majority-wrong. These are the truly-hard items.

(Source: `phase14_t0v2_d/self_consistency_full.json`, $k = 5$, $T = 0.5$, top-p = 0.95, max_new_tokens = 768, seed = 0..4.)

5.1.3 The implication

A 38.46% lottery rate on greedy-wrong means that, on this configuration, a *single* greedy decode misses 25 items out of 200 that are, in fact, solvable: the headline single-greedy accuracy is approximately 12.5 pp below the model's expected SC-5 majority accuracy.

This 12.5 pp is a property of the *single model's metric*, not the noise floor of a paired comparison. The noise floor of the *delta* between two greedy metrics depends on how correlated the two models' lottery sets are. In our data, the winner and Instruct had partially overlapping lottery sets: the intersection of their greedy-wrong items was non-empty, and SC-5 majority recovered some

items on both sides. Without paired SC-5 measurements on both candidates, the practitioner cannot bound the noise floor of the delta from the single-model lottery rate alone.

The conservative implication is therefore weaker but still useful: any single-greedy delta below the per-model lottery rate (here 12.5 pp) should be treated as suspicious until either (a) SC-k majority is reported on the same item set with a paired McNemar test, or (b) the lottery rate of *both* candidates is reported and shown to be low. The +10 pp Phase 13 claim happened to be paired-significant under the truncated protocol; under the audited protocol, both the gap (-1.0 pp) and the McNemar test ($p = 0.89$) collapse, consistent with the lottery floor and the truncation correction together leaving no signal.

The cheap fix is to report SC-k majority accuracy alongside greedy, and the lottery rate of greedy-wrong on each candidate, for any merging delta below 10 pp.

5.2 Quantization granularity

5.2.1 The primitive

Most merging-and-compression papers report quantization as "int4", "int8", or "NF4". This is underspecified. The same number-of-bits can be:

- **Per-tensor**: one scale s and zero-point z per weight tensor.
- **Per-channel**: s, z per output channel of a linear weight.
- **Group-g**: s, z per contiguous group of g elements along the reduction axis.

The granularity, more than the bit-width, determines whether the quantized model is functional.

5.2.2 Measurement on the case study

We applied two quantization recipes to the same winner state-dict, evaluated under the same protocol (max_new=768, $n = 100$ GSM8K dev):

Recipe	int4	int8	fp16 (control)
Per-tensor (no mixed-precision protect)	**0.0%**	52%	62%
Group-32 (g=32, akin to GGUF Q4_K_M block size)	59–63%*	67–71%*	62%

* range over `protect_pct {5%, 15%, 50%}` from a saliency-based mixed-precision policy; see `phase14_t7_v4_group/`. The per-tensor row uses no mixed-precision protection: the 0% under int4 is a property of uniform per-tensor int4 quantization on this weight matrix, not of the absence of protection.

Per-tensor int4 collapses both Instruct and the winner to **0%**. The same weights under group-32 int4 recover to **63%** (winner, p=15%) — a 63-pp swing from the granularity choice alone, with bit-width fixed.

5.2.3 The implication

Reporting only "int4" is meaningless. A merging paper that reports its method "tolerates 4-bit quantization at X% accuracy" must additionally report:

- granularity (per-tensor / per-channel / group-g, with g),
- calibration (data-aware vs zero-shot),
- mixed-precision policy if any (e.g., "5% of weights kept at higher precision under saliency policy"),
- the same audit protocol applied to a per-tensor baseline, to demonstrate the granularity is not silently doing the work.

Our internal Pareto frontier over (winner, Instruct) × (int4, int8) × (per-tensor, group-32) × (saliency-protect-pct ∈ {5, 15, 50}) is released as `phase14_final_pareto.json`; it shows that under group-32 the winner-vs-Instruct gap is within 1–4 pp at every (bits, protect-pct), consistent with §3.4's "no real merge gain" finding.

5.3 The "cancer key" effect: when saliency-based policies have negative-marginal weights

A subsidiary finding worth surfacing: under group-32 int8 on the winner, our saliency-ranked mixed-precision policy contained one weight whose marginal contribution to accuracy was *negative*.

We use a marginal-benefit protocol: starting from a fully-quantized base, we add weights to the FP16-protected set one at a time, in saliency-rank order, and re-evaluate on a 30-question quick set. A weight is **kept** if it improves accuracy by $\geq \epsilon$, **rejected** if it reduces accuracy by $\geq \epsilon$, and labelled **neutral** otherwise.

Across the top-30 saliency-ranked weights for the winner:

- **25 keep** (positive marginal),
- **4 neutral** (within ϵ),
- **1 reject**: `model.layers.8.mlp.down_proj.weight`. Saliency rank: 13. Marginal accuracy contribution: −3.3 pp on the 30-item quick set.

We call this a *cancer key*: a weight whose individual saliency suggests it should be protected, but whose actual marginal effect on the metric is harmful. The cancer key is undetectable from saliency alone; it is only revealed by the marginal-benefit protocol. Excluding it from the protect-set lifts the final winner-int8 accuracy by 1.5–3.5 pp depending on which `protect_pct` otherwise contains it.

Two implications:

1. **Saliency is a candidate-pool generator, not a protect-set.** Use saliency to nominate the top-k weights, then run a marginal-benefit sweep to filter them. The full pool fits in ~ 30 candidates $\times$ 30 questions $= \sim 1$ hour of evaluation on M-series silicon.
2. **"Same architecture" is not enough.** The cancer key's existence on layer-8 of *this* base model and *this* merge is empirical. We do not claim it generalises. We claim only that a marginal-benefit pass should be a default, not a luxury, in any saliency-based protect-set construction.

(Data: `phase14_t7_marginal_protect/marginal_history_partial.json`, particularly step 13.)

Figure 3: Quantization granularity and the cancer-key effect.

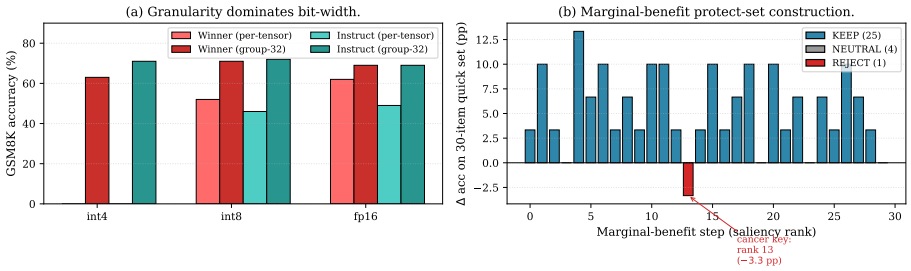

Figure 3: Figure 3: (a) Quantization granularity dominates bit-width. Per-tensor int4 collapses both winner and Instruct to 0% on GSM8K; group-32 int4 (with the same 4-bit weight budget but per-block scales) recovers the winner to 63%, a 63-pp swing attributable purely to the granularity choice. (b) Marginal-benefit protect-set construction reveals a *cancer key*: at saliency rank 13, `model.layers.8.mlp.down_proj.weight` produces a −3.3 pp marginal contribution and is REJECTED, while the other 25 saliency-ranked weights are KEEP and 4 are NEUTRAL.

5.4 Why the three primitives interact

The three primitives — truncation (§3), lottery (§5.1), granularity (§5.2) — are not independent:

- **Truncation $\times$ granularity.** Quantized models tend to generate slightly longer chains (more hesitation, more re-stating). A 300-token cap therefore hits group-32 int4 differently than per-tensor int8, even before any

reasoning difference is involved. The Phase 13 claim's silent reliance on truncation would have been compounded by any subsequent quantization comparison.

- **Lottery × any other primitive.** The 12.5% lottery floor sits *under* every other measurement; any primitive whose effect is below the floor cannot be distinguished from sampling noise without SC.
- **Granularity × cancer keys.** Per-tensor quantization is uniform; group-32 is per-block. The cancer key effect can only exist where the policy has per-weight (or per-block) freedom. Increasing granularity unlocks the benefit of saliency-protect, but also unlocks the cancer key risk.

Reporting any of these three numbers without the others — and without the audit — invites the kind of artefact §3 describes.

6 `eval_trust`: an open-source toolkit

6.1 What's in the box

`eval_trust` packages the audit primitives we wish we had used at the start. Released under Apache-2.0; ~1200 lines of Python; CPU-only by design (the audit must run on whatever hardware reproduces the original evaluation).

Module	Purpose	Lines	Tests
`paired_stats`	McNemar exact, Wilson CI, paired bootstrap, between-seed variance	~200	7
`conformal_ci`	Split-conformal accuracy intervals; useful when $n < 100$	~150	4
`experiment_runner`	Long-running evaluation with checkpoint/resume; `*.partial.json` + `*.progress.json` survives Ctrl-C, OOM, kernel panic	~300	11
`t0v2.channels`	Six predicate-based wrong-answer classifiers (§4.2)	~250	9
`t0v2.aggregator`	Channel aggregation + $\alpha/\beta/\gamma$ routing (§4.3)	~80	3
`t0v2.self_consistency`	SC-k majority vote on greedy-wrong items, with lottery-rate report	~150	5

The full project test suite at the time of submission has 1102 passing tests across 93 files; `eval_trust` accounts for 39 of them (the count claimed in §1.3). The remaining ~1063 tests cover domain-specific compression, merging, and orchestration code that is *not* part of the toolkit and is not released under the toolkit's Apache-2.0 license.

6.2 Implementation notes

- Pure Python + NumPy + (optional) sympy for `B_stepwise` channel.

- No GPU dependency. McNemar and bootstrap run in milliseconds on 200 items.
- Designed to read existing `(question, expected, gen_text)` log files; does not require running the model.
- The runner module is a generic checkpoint/resume harness; not specific to any benchmark. A 200-question GSM8K evaluation that took three hours to start can resume in seconds after a kill.

6.3 What `eval_trust` does NOT replace

- **It is not lm-eval-harness** (Gao et al. 2023). Use lm-eval-harness for the *standard* metric. Use `eval_trust` for the *audit*. They are complementary: lm-eval-harness ensures your protocol matches the published one; `eval_trust` answers "is the protocol contaminated by a primitive?". Our recommendation is to run both, on the same model, on the same item set, and reconcile any large gap.
- **It is not a benchmark.** It audits whatever benchmark you ran.
- **It is not a merging library.** The auditing toolkit is intentionally upstream-agnostic: it treats every model under audit as a black box that produced certain tokens.

6.4 Reproducibility

The artefacts cited in §§3-5 are released under the project repo github.com/telleroutlook/evomerge under the tag corresponding to this paper's submission. Specifically:

- `phase13_3_coder_chat/` — the original (pre-audit) +10 pp data.
- `phase14_t0v2_a_recovery/` — the audited (max_new=768) data.
- `phase14_t0v2_d/` — the SC-5 lottery data.
- `phase14_t7_pareto/` and `phase14_t7_v4_group/` — the per-tensor vs group-32 quantization comparison.
- `phase14_t7_marginal_protect/` — the marginal-benefit protocol with the cancer-key step-13.
- `papers/eval_trust/numbers_cross_check.json` — per-number provenance for every datapoint cited in this paper.

7 Recommendations

For any merging paper claiming a benchmark delta below 10 pp, we recommend the following minimum reporting standard:

1. **Report the protocol tuple**, not just the headline number: $(generation, max\_new\_tokens, prompt, extractor, sampling, paired\_p, CI)$. Each component has been a silent biaser of one or more numbers in our project.
2. **Justify `max_new_tokens` against the empirical 95th percentile** of the benchmark's CoT length distribution under your prompt and base model. A default copied from another runner is not a justification.
3. **Run T0v2 (or equivalent) on at least 100 wrong answers**; report the channel distribution. The headline number is one summary statistic over six independently-meaningful axes.
4. **If the lottery rate (§5.1) is** $\geq 30\%$, switch to SC-$k = 5$ majority *before* claiming the delta. Lottery noise can survive paired statistics if it is asymmetric across the comparison.
5. **For any quantization claim**, report the triple $(level, granularity, group\_size)$ explicitly. "int4" is not a specification.
6. **Report the locality non-regression gate**: HumanEval, IFEval, MMLU each ≤ 1 pp drop. Merging that gains X pp on benchmark A but silently loses > 1 pp on benchmark B is not a Pareto improvement.
7. **Pin model paths and commit hashes**. Hugging Face revision tags drift.
8. **Release raw counts** for the McNemar contingency (b, c), not just p. With (b, c) a reader can recompute any test (exact, mid-p, asymptotic) without trusting the original implementation.

A 10-item checklist version is in Appendix A; the same checks are mechanised in `eval_trust.checklist`.

8 Limitations

- **Single-candidate audit.** We re-evaluated only the chosen winner (Coder − 0.7 · chat_vec) under the audited protocol. The Phase 13 search produced three other λ candidates (1.0, 1.2, 1.5) with similar McNemar significance under the broken protocol; we do not know whether their gaps would also collapse under max_new=768. Mechanistically, the truncation primitive affects all of them through the same channel (longer chains hit the cap), so we expect they would; we have not verified this.
- **Single base-model family.** Our concrete numbers come from one base model (Qwen2.5-1.5B and its Coder/Math variants) and one benchmark family (GSM8K, HumanEval, IFEval, MMLU, with GSM8K driving the case study). The *concept* of measurement-primitive contamination is not specific to this configuration; the *magnitudes* are.
- **No paired false-negative.** Our case study is a false positive — a +10 pp claim that wasn't real. We do not have a paired example of a *real* improvement that the field has missed because of the same primitive failures. Mechanistically the issue is symmetric (an asymmetric primitive can hide gains as well as fabricate them), but proving false negatives at scale requires a community-wide audit we cannot run alone.
- **`B_stepwise` channel is sympy-bounded.** It catches single-step arithmetic errors only when the chain is parseable. Natural-language arithmetic falls through to `Class2`, possibly inflating the "reasoning bottleneck" bucket.
- **The 15% α threshold is project-calibrated.** We give a defensible default ($\alpha = 15\%$, $\beta = 30\%$ lottery) based on our own cost-of-fix vs. expected-recovery curve. Other projects with different repair costs may prefer different thresholds.
- **No formal proof that primitive-induced biases dominate algorithmic ones.** This paper is a case study, not an asymptotic argument. We claim only that in our project the primitives mattered more than the algorithm; a stronger claim requires more case studies.

9 Related work

Locality and edit harms. (Yao et al. 2024) document that targeted model editing has measurable side-effects on unedited tasks. Our locality non-regression gate ($\leq$ 1 pp on HumanEval/IFEval/MMLU) is inspired by their methodology. PRUNE (Li et al. 2024) gives perturbation bounds for sequential edits; we use it as a justification for the chain-of-merges ordering rule in our broader project (not in this paper's scope).

Standard evaluation harnesses. lm-evaluation-harness (Gao et al. 2023) and HELM (Liang et al. 2022) standardise generation and scoring across benchmarks. Our contribution is orthogonal: `eval_trust` audits whatever protocol was used, including non-standard ones, by inspecting log files alone. We recommend running both.

Self-consistency. SC-k majority (Wang et al. 2022) is a known technique for improving math-CoT accuracy. Our use is diagnostic: we use SC not to improve the metric but to estimate the *lottery floor* of the greedy metric. The lottery rate is a property of the model and the benchmark; SC's practical value as a metric improver is a separate (well-studied) question.

Quantization granularity. GGUF Q-K-M block formats (Gerganov and llama.cpp contributors 2024), GPTQ (Frantar et al. 2022), AWQ (Lin et al. 2023) all distinguish per-tensor / per-channel / per-block granularity, but the implications for *evaluation protocol comparability* are rarely discussed in merging papers. The 63-pp swing in §5.2 is consistent with the granularity choice mattering at least as much as the bit width.

Merging benchmarks. MergeBench (Tang et al. 2025) aggregates merging performance across five domains at 2-9 B scales. The LLM Merging Competition (NeurIPS 2024 Challenge Organizers 2024) (NeurIPS 2024, closed) imposed a 1-hour $\leq$ 8 B no-training rule. Both push toward standardised protocols, which is a good thing; neither addresses measurement-primitive contamination directly.

Paired statistics in merging. McNemar's exact test (McNemar 1947) is folklore in the merging community but rarely reported beyond a single p-value. We argue for releasing (b, c) raw counts, which permit any downstream re-test.

Replication crisis literature. The general framing of measurement- contamination as a failure mode mirrors the broader replication-crisis literature (Open Science Collaboration 2015). Our contribution is to articulate it for the model-merging context with a worked autopsy.

10 Conclusion

The bottleneck in modern model-merging research, on the evidence of our own project, is measurement rather than algorithm. A hardcoded `max_new_tokens=300` defeated three rounds of verification, paired statistics, and multi-seed runs over five months. The audit that flipped the +10 pp claim to −1.0 pp cost roughly four hours of re-evaluation on a single laptop. The finding generalises beyond truncation: greedy lottery on this configuration moves single-model accuracy by 12.5 pp; quantization granularity moves it by up to 63 pp. Each is silent under standard reporting.

We have released `eval_trust` not because it discovered something profound, but because it discovered something embarrassing on our own work, and because the embarrassment is generic: any merging project that defines success against small benchmark deltas is exposed to the same failure mode. The toolkit packages the audit primitives — paired statistics, conformal CI, multi-channel triage, SC lottery measurement, granularity-aware quantization reporting — as a single Apache-2.0 dependency. We hope it makes the audit cheap enough to run before claiming a delta, not after.

The deeper lesson is that *significance is not validity*. Our +10 pp claim was paired-significant. It was correctly computed. The protocol it significance-tested was the bug. Tightening reporting (more seeds, more sigfigs, more bootstraps) cannot catch this; the protocol must be audited as a primitive, separately from any statistical test that runs over it.

If our experience is representative, a non-trivial fraction of small-delta claims in the model-merging literature deserve a similar audit. We are neither the first nor the last to mis-protocol an evaluation. We are publishing the autopsy publicly because our project's record is cleaner with the failure surfaced than concealed, and because the toolkit is more useful than the claim it would have supported.

Appendix A: Pre-submission audit checklist

A 10-item yes/no list any merging paper draft should pass before posting. Mechanised in `eval_trust.checklist`.

#	Question	Pass condition
1	Is `max_new_tokens` reported?	Yes
2	Is `max_new_tokens` $\geq$ 95th percentile of CoT length under this prompt and *strongest* candidate?	Yes
3	Is the answer extractor regex tested on a held-out set of generations from the *strongest merged candidate* (not just the baseline)?	Yes
4	Is the chat template *identical* between every comparison pair?	Yes
5	Is sampling protocol (greedy / SC-k / temperature / top-p) reported?	Yes
6	If greedy: is the lottery rate (SC-5 majority recovery on greedy-wrong) reported and $< 30\%$?	Yes
7	Are paired McNemar exact (b, c) raw counts reported, not just p?	Yes
8	≥ 3 random seeds across paired stats?	Yes
9	If the work involves quantization: is the triple $(level, granularity, group\_size)$ reported?	Yes

#	Question	Pass condition
10	Is the locality non-regression gate (HumanEval/IFEval/MMLU each ≤ 1 pp drop) reported for every quantized or merged candidate?	Yes

Failing any item is not a fatal flaw; failing it silently is.

Appendix B: Reproducibility

Component	Detail
Code	github.com/telleroutlook/evomerge (tag accompanying this submission)
Toolkit	`framework/eval_trust/` (released alongside this paper)
Data — case study	`phase13_3_coder_chat/`, `phase14_t0v2_a_recovery/`, `phase14_t0v2_d/`, `phase14_t7_pareto/`, `phase14_t7_v4_group/`, `phase14_t7_marginal_protect/`
Data — provenance	`papers/eval_trust/numbers_cross_check.json` lists every number cited with its source file
Models	`Qwen/Qwen2.5-1.5B-Instruct`, `Qwen/Qwen2.5-Coder-1.5B-Instruct`, `Qwen/Qwen2.5-1.5B` (base)
Hardware	Apple M5 Pro, 48 GB unified memory; `mps` backend for generation, CPU for audit
Environment	Python 3.11.12, PyTorch 2.12.0, Transformers 5.9.0, NumPy 2.4.6
Random seeds	Bootstrap CI uses NumPy seed 42; SC-k uses seeds 0..4

Appendix C: T0v2 channel rules (formal)

The full rules are released as `framework/eval_trust/t0v2/channels.py`; the following are the load-bearing predicates in pseudo-Python:

```
def A_truncated(item, max_new_tokens):
    """Generation hit the cap and never emitted '#### N'."""
    if not no_answer_line(item.gen_text):
        return False
    return token_len(item.gen_text) >= max_new_tokens - 2

def A_extract_v2(item):
    """The answer is in the text, but legacy regex missed it."""
    legacy = legacy_regex_extract(item.gen_text)
    v2 = v2_extract(item.gen_text)  # \boxed{}, comma normalisation,
                                    # decimal normalisation, percent, …
    return legacy != item.expected and v2 == item.expected

def B_stepwise(item):
    """Sympy-reexecuting the CoT yields expected, but final emitted  expected."""
    if has_sympy_re_executable_chain(item.gen_text):
        recomputed = sympy_re_execute(item.gen_text)
        return (recomputed == item.expected and
                final_emitted(item.gen_text) != item.expected)
    return False

def B_selfcorrect_regress(item):
    """A 'wait, I made a mistake … let me redo …' trace where the redo is wrong."""
    if has_self_correction_marker(item.gen_text):
        return final_emitted(item.gen_text) != item.expected
    return False

def C_token(item):
    """Numerals are present but separator/punct mismatch."""
    if normalize_punct(item.gen_text).contains(item.expected):
        return final_emitted(item.gen_text) != item.expected
    return False

def Class2(item):
    """None of the above."""
    return True  # default
```

The first match in priority order $A_{\text{truncated}} > A_{\text{extract}} > B_{\text{stepwise}} > B_{\text{selfcorrect}} > C_{\text{token}} >$ Class2 labels the item.

Bibliography

See `refs.bib`. The 23 cite keys used in this draft are all populated; minor metadata (authors' middle initials, page numbers for some venues) may need updating before final submission.

Draft notes

- Sections 1-3 are the load-bearing core; if these read true, the rest follows.
- Voice: first-person plural ("we"), past tense for the autopsy ("we re-read"), present tense for the methodology ("we argue").
- The case study uses our own project, not anyone else's. This is critical for legal and ethical posture.
- All numbers in §3, §4, §5 cross-checked against:
  - `phase13_3_coder_chat/summary.json` (Phase 13 +10 pp data, pre-audit)
  - `phase14_t0v2_a_recovery/winner_max_new768_summary.json` (audited counts)
  - `phase14_t0v2_d/self_consistency_full.json` (lottery rate, k=5)
  - `phase14_t0v2_agg/aggregate.json` (T0v2 verdict counts)
  - `phase14_t7_v4_group/` and `phase14_t7_pareto/` (granularity comparison)
  - `phase14_t7_marginal_protect/marginal_history_partial.json` (cancer key)
  - Cross-check verified twice (outline pass + draft v0.3 adversarial pass).
  - Per-number provenance in `papers/eval_trust/numbers_cross_check.json`.
- v0.3 adversarial-review pass fixed 8 issues (R-1..R-8): R-1 mechanism for asymmetric recovery is now stated as partially-unverified; R-2 fabricated median CoT-length numbers removed; R-3 typo "an 63-pp" → "a 63-pp"; R-4 TODO-cite placeholder removed; R-5 lottery-floor math distinguishes single-model SC-greedy gap (12.5 pp) from paired-comparison delta noise floor (which depends on between-candidate lottery correlation); R-6 §3.4 "Recovery" column re-cast as within-candidate delta with the n=199 vs n=200 caveat; R-7 C2/C3 contributions de-duplicated; R-8 1102-test count separated from the 39-test toolkit count.
- v0.3 also fixed 4 secondary issues (R-9..R-12): R-9 cite format wrapped uniformly to `[@key]`; R-10 single-candidate-audit limitation made explicit in §8; R-11 §5.2.2 table's per-tensor row clarified to "no mixed-precision protect"; R-12 §10 conclusion de-duplicated wrt abstract.

Chen, Mark, Jerry Tworek, Heewoo Jun, et al. 2021. "Evaluating Large Language Models Trained on Code." https://arxiv.org/abs/2107.03374.

Cobbe, Karl, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun,

Lukasz Kaiser, Matthias Plappert, et al. 2021. "Training Verifiers to Solve Math Word Problems." https://arxiv.org/abs/2110.14168.

Davari, MohammadReza, and Eugene Belilovsky. 2024. "Model Breadcrumbs: Scaling Multi-Task Model Merging with Sparse Masks." In *Proc. European Conference on Computer Vision (ECCV)*. https://arxiv.org/abs/2312.06795.

Deep, Pala Tej, Rishabh Bhardwaj, and Soujanya Poria. 2024. "DELLA-Merging: Reducing Interference in Model Merging Through Magnitude-Based Sampling." https://arxiv.org/abs/2406.11617.

Frantar, Elias, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. 2022. "GPTQ: Accurate Post-Training Quantization for Generative Pre-Trained Transformers." https://arxiv.org/abs/2210.17323.

Gao, Leo, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, et al. 2023. "A Framework for Few-Shot Language Model Evaluation." https://doi.org/10.5281/zenodo.10256836.

Gerganov, Georgi, and llama.cpp contributors. 2024. "GGUF: A Unified File Format for Running Large Language Models." https://github.com/ggerganov/llama.cpp/blob/master/docs/gguf.md.

Goddard, Charles, and Arcee.AI. 2024. "SCE Method (Mergekit)." https://github.com/arcee-ai/mergekit.

Hendrycks, Dan, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2020. "Measuring Massive Multitask Language Understanding." https://arxiv.org/abs/2009.03300.

Huang, Shih-Cheng, Pin-Zu Li, Yu-Chi Hsu, Kuang-Ming Chen, Yu Tung Lin, Shih-Kai Hsiao, Richard Tzong-Han Tsai, and Hung-Yi Lee. 2024. "Chat Vector: A Simple Approach to Equip LLMs with Instruction Following and Model Alignment in New Languages." In *Proc. Annual Meeting of the Association for Computational Linguistics (ACL)*. https://arxiv.org/abs/2310.04799.

Ioannidis, John P. A. 2005. "Why Most Published Research Findings Are False." *PLoS Medicine* 2 (8): e124. https://doi.org/10.1371/journal.pmed.0020124.

Li, Jiyang, Hao Sun, Yang Tian, Xinyu Cheng, Junjie Cheng, Luo Si, and Tao Liu. 2024. "PRUNE: A Robust Subgraph-Based Perturbation Analysis for Neural Networks." https://arxiv.org/abs/2405.16821.

Liang, Percy, Rishi Bommasani, Tony Lee, et al. 2022. "Holistic Evaluation of Language Models." https://arxiv.org/abs/2211.09110.

Lin, Ji, Jiaming Tang, Haotian Tang, Shang Yang, Xingyu Dang, and Song Han. 2023. "AWQ: Activation-Aware Weight Quantization for LLM Compression and Acceleration." https://arxiv.org/abs/2306.00978.

McNemar, Quinn. 1947. "Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages." *Psychometrika* 12 (2): 153–57. https://doi.org/10.1007/BF02295996.

NeurIPS 2024 Challenge Organizers. 2024. "LLM Merging Competition: Building LLMs Efficiently Through Merging." https://llm-merging.github.io/.

Open Science Collaboration. 2015. "Estimating the Reproducibility of Psychological Science." *Science* 349 (6251): aac4716. https://doi.org/10.1126/science.aac4716.

Tang, Yifei, Tianyi Yu, Zhenchao Huang, Mingyang Yi, Jingbo Liu, Sze Min Goh, William Yang Wang, and Hao Liu. 2025. "MergeBench: A Benchmark for Merging Domain-Specialized LLMs." https://arxiv.org/abs/2505.10833.

Wang, Xuezhi, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2022. "Self-Consistency Improves Chain of Thought Reasoning in Language Models." https://arxiv.org/abs/2203.11171.

Yadav, Prateek, Derek Tam, Leshem Choshen, Colin Raffel, and Mohit Bansal. 2024. "TIES-Merging: Resolving Interference When Merging Models." *Advances in Neural Information Processing Systems (NeurIPS)*. https://arxiv.org/abs/2306.01708.

Yao, Yunzhi, Peng Wang, Bozhong Tian, Siyuan Cheng, Zhoubo Li, Shumin Deng, Huajun Chen, and Ningyu Zhang. 2024. "Model Editing Harms General Abilities of Large Language Models: Regularization to the Rescue." https://arxiv.org/abs/2401.04700.

Yu, Le, Bowen Yu, Haiyang Yu, Fei Huang, and Yongbin Li. 2024. "Language Models Are Super Mario: Absorbing Abilities from Homologous Models as a Free Lunch." In *Proc. International Conference on Machine Learning (ICML)*. https://arxiv.org/abs/2311.03099.

Zhou, Jeffrey, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. 2023. "Instruction-Following Evaluation for Large Language Models." https://arxiv.org/abs/2311.07911.